

Hands-On Object-Oriented Programming

SWEN1 | BIF3

OOP - Why?

- OOP is fast and easy to execute
- OOP provides a clear structure for the programs
- OOP helps to keep the C# code DRY "Don't Repeat Yourself", and makes the code easier to maintain, modify and debug
- OOP makes it possible to create full reusable applications with less code and shorter development time

Tip: The "Don't Repeat Yourself" (DRY) principle is about reducing the repetition of code. You should extract out the codes that are common for the application and place them at a single place and reuse them instead of repeating it.

Naming conventions in C#

```
class ClassWithMembers {
  // constant values are not changeable
  public const float ConstantValue = 3.14f;
  // static values are the same value over all instances
  // ATTENTION: also changeable in non-static methods
  public static string StaticString = "Value";
  // readonly can only be changed in constructor
  public readonly int ReadOnlyInt = 0;

  // full auto property
  public int IntProperty { get; set; }
  // property with private setter and default value
  public int ReadProperty { get; private set; } = 0;

  // only accessable in derived classes
  protected string _protectedString = "default";

  // only accessable in this class
  private int _privateVariable = 7;

  // constructors
  public ClassWithMembers() { }

  // overloaded constructor
  public ClassWithMembers(int x, int y) {
      ReadOnlyInt = _privateVariable;
      ChangeValue(x, y);
  }

  // public member methods
  public void ChangeSomeValues(int newValue) {
      _privateVariable = newValue;
      ReadProperty = newValue;
  }

  public void WriteSomething() {
      Console.WriteLine("Write Something");
  }

  private void ChangeValue(int x, int y) {
      _privateVariable = x + y;
  }
}
```

Naming conventions in Java

```java
public class ClassWithMembers {
  // constant values are not changeable
  public static final float CONSTANT_VALUE = 3.14f;
  // static values are the same value over all instances
  public static String STATIC_STRING = "Value";
  // final can only be set here or in constructor
  public final int readOnlyInt = 0;

  // property with getters & setters
  private int intProperty;
  public int getIntProperty() {
    return intProperty;
  }
  public void setIntProperty(int intProperty) {
    this.intProperty = intProperty;
  }

  // full auto property - thx to lombok-library
  @Getter @Setter
  private int nicerIntProperty;

  // property without setter but with default value
  private int readProperty;
  public int getReadProperty() {
    return readProperty;
  }

  @Getter // auto property with lombok
  private int nicerReadProperty = 0;
  // only accessable in derived classes
  protected String protectedString = "default";
  // only accessable in this class
  private int privateVariable = 7;

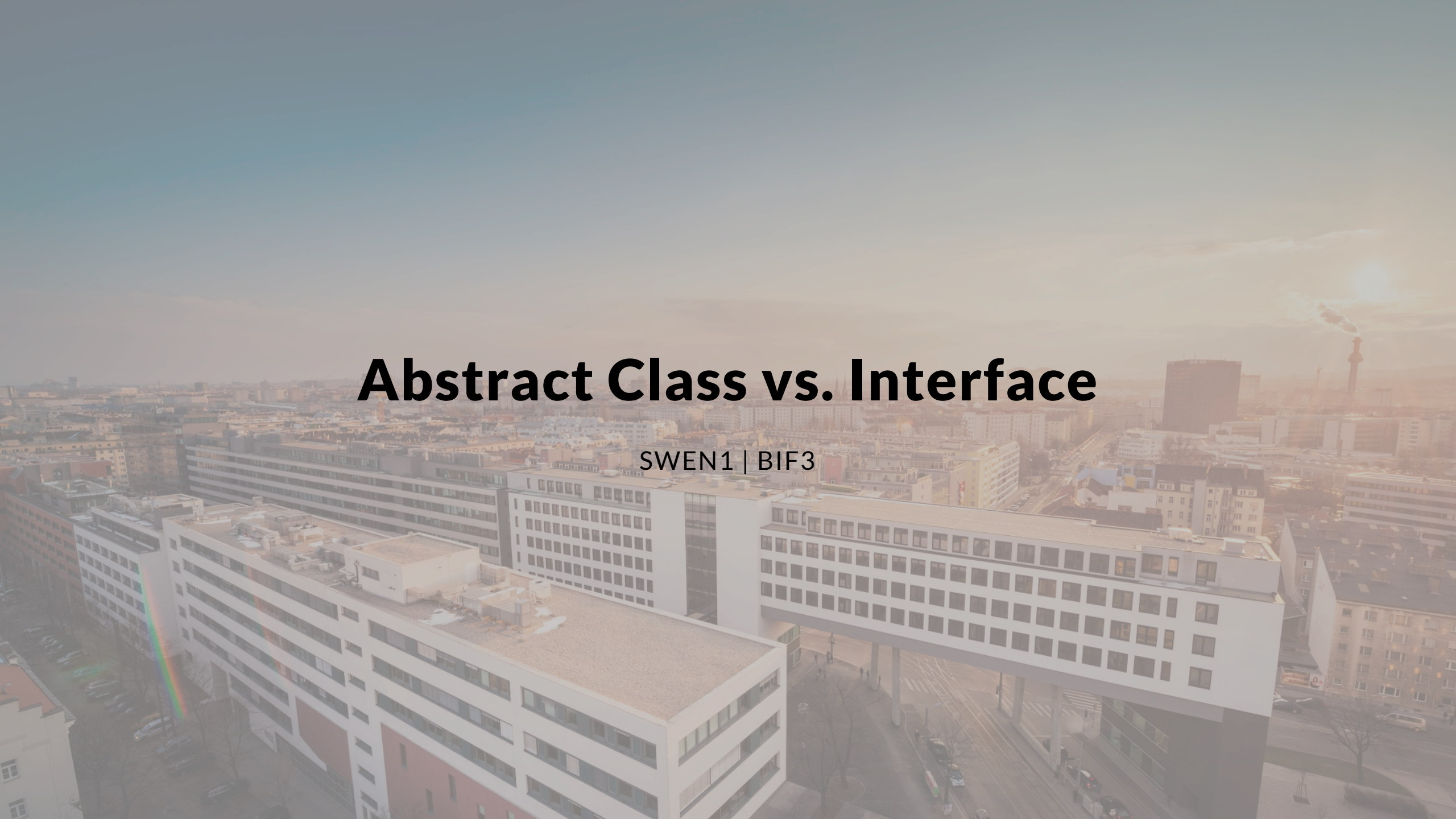
  // default constructor
  public ClassWithMembers() {
    readOnlyInt2 = 0;
  }

  // overloaded constructor
  public ClassWithMembers(int x, int y) {
    readOnlyInt2 = privateVariable;
    changeValue(x, y);
  }

  // public member methods
  public void changeSomeValues(int newValue) {
    privateVariable = newValue;
    readProperty = newValue;
    // or introduce and use
    // private void setReadProperty(...)
  }

  public void writeSomething() {
      System.out.println("Write Something");
  }

  private void changeValue(int x, int y) {
      privateVariable = x + y;
  }
}
```



Abstract Class vs. Interface

SWEN1 | BIF3

Abstract Class vs. Interface

```
abstract class Shape {
  public abstract void Draw();
  public void Move() {...}
}

class Line : Shape {
  public override void Draw() {
    ...
  }
}
```

```
public interface I {
  void F(int i);
  object P { get; set; }
}

class A : I, ICloneable {
  public void F(int i) {...}
  public object P {
    get {...} set {...}
  }
  public object Clone() {...}
}
```

- Classes with abstract methods have to be abstract
- Abstract methods are implicitly virtual
- Abstract classes can NOT be instantiated

- Interfaces cannot have fields
- Methods of an interface are public abstract
- Classes can implement multiple interfaces

Important: Changing (abstract) base classes should NOT affect the functionality of derived classes!

When to use Abstract Class or Interface?

Abstract Class

When we need to implement a base type that knows how to implement some part of the abstraction i.e. partial implementation and the other/remaining part of the abstraction cannot be implemented by this class.

-> What kind of object is it?

Interface

When we only need to enforce a contract. Whoever will implement this interface will provide an implementation of the methods.

--> What should the object be capable of?

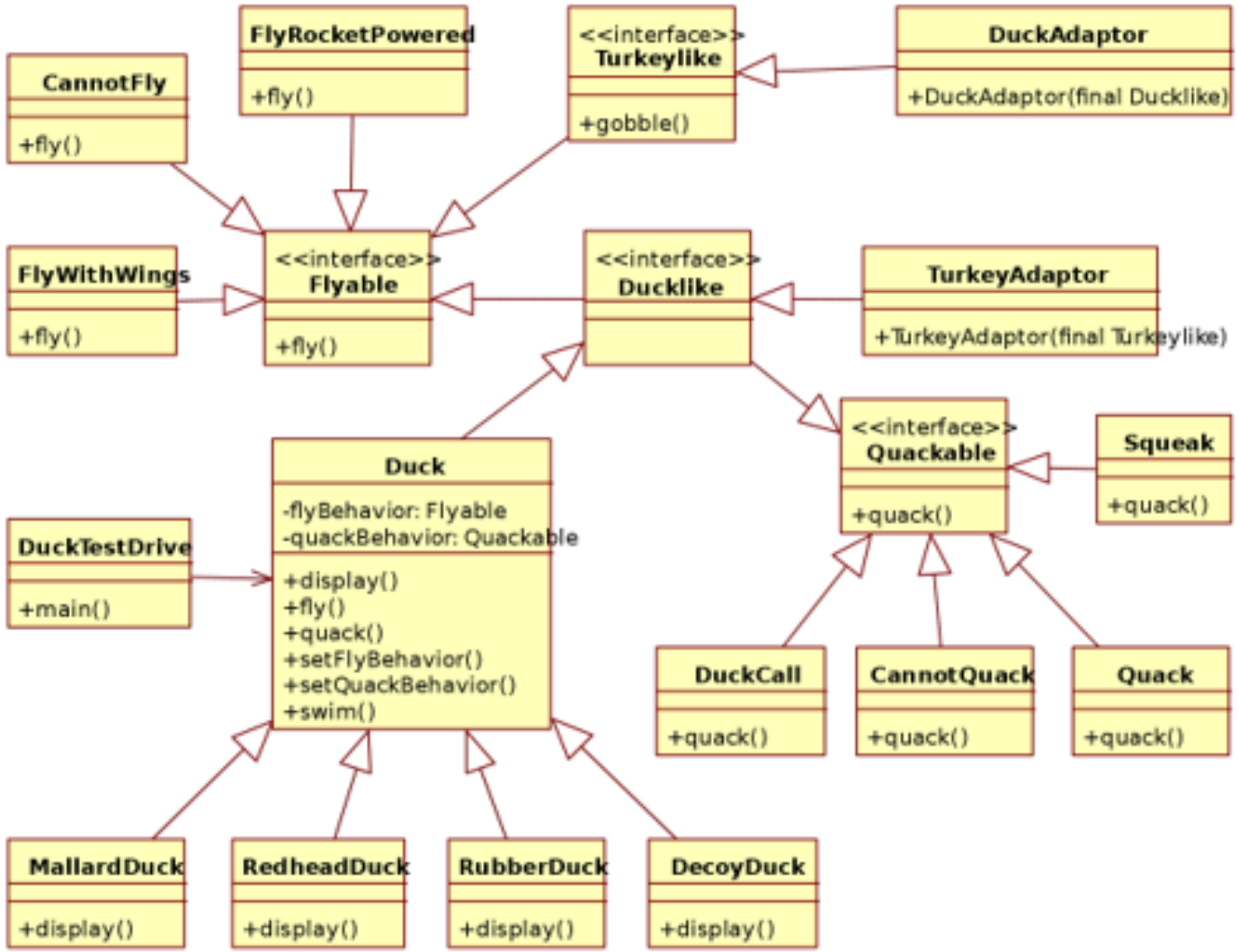
Abstract Class vs Interface Conclusion

- **Concrete class** is the actual implementation.
- **Interface** is the contract, it specifies the abstraction contract that its implementer should implement.
- **Abstract class** is a trade off between the two i.e. when we need to have the partial implementation of the contract then we can use abstract class.

Recommendations:

- Program to interfaces, not implementations!
  --> loose coupling, maintainability, scalability, accessibility
- YAGNI: Always check if abstract classes/interfaces are useful in the particular use case. Don‘t overdo it.
  --> Just keep it simple!

Composition over Inheritance





Object-Oriented Programming by Example

SWEN1 | BIF3

Value- and Reference-Types

Value Types

- allocated at stack or around the object
- Default value is `0`, `'\0'` or `false`
- Value will be **copied** when being assigned
- It's possible to declare custom value types in C# (`enum` and `struct`)

Reference Types

- Will be allocated on heap
- Default value is `null`
- When being assigned, the reference to the object will be copied, not the object itself.

Boxing and Unboxing

Boxing

Value type --> reference type
Boxing is an implicit operation if an object is needed, but a value is provided.

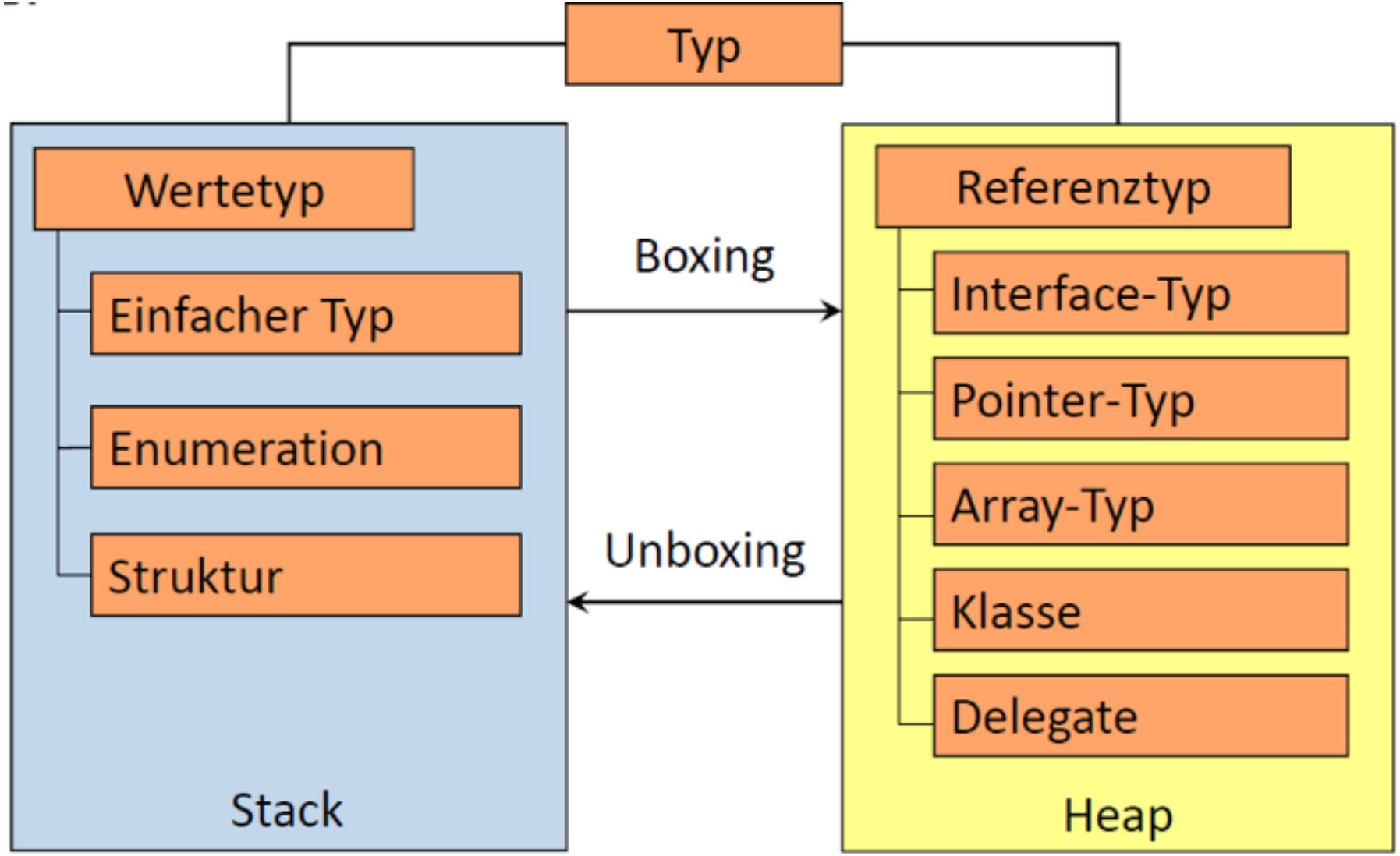
```
Console.WriteLine(string.Format("i={0}", i));
string s = 123.ToString();
```

Unboxing

Reference type --> value type
An explicit cast is needed for unboxing.

```
int i = 123;
object o = i; // boxing
int j = (int)o; // unboxing
```

Summary – Value and Reference Types



Polymorphism

The course project contains different implementations (residing in different folders) of a simple "Shape" library, to illustrate different approaches for possible implementations.
Please note, that the way of implementing such a "library" may vary from project to project.

- `AbstractBaseClass`: Implemented only with inheritance and method overriding
- `InterfaceBasedShapes`: Shape library which only relies on interfaces
- `OptimizedSolution`: A combination from interfaces with an abstract base class

Analysis of the "Shape Library" project

- There is no possibility to easy extend classes with functionality when using the **AbstractBaseClass** sample, as Java and C# do NOT support multi-inheritance. The only way to add additional functionality is by extending the base class, which often leads to bloat code, code duplication.
- When using **InterfaceBasedShapes** it‘s easy to extend classes with additional interfaces. But every class has to take care of its own origin coordinates, which again leads to code duplication.
- For our use-case "Shape Library" the **OptimizedSolution** is a very elegant solution. The abstract base class tells *what kind of object* it is (`Shape`) and *what it should be capable of* (`IShapeComposition` – a combination of `IShape` and `IShapeMath`). The hierarchy of inheritance stays small (rare usage of Multilevel-Inheritance) and objects are easy to extend (just add another interface).

Links:

- Java: https://git.technikum-wien.at/swen/swen1/java/shapelibdemo
- C#: https://git.technikum-wien.at/swen/swen1/csharp/shapelibdemo